Data Structure

Lecture 9:
Searching and Sorting

Asst. Instructor
Ali A. Al-Ani

1

Searching

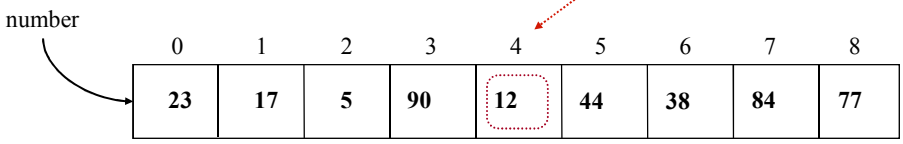
- **Searching** is the process of finding a target element among a group of items (the search pool), or determining that it isn't there.
- Here's the problem statement:
  - Given a value X, return the index of X in the array, if such X exists. Otherwise, return NOT_FOUND (-1). Assume there are no duplicate entries in the array.
- The number of comparisons the algorithms make to analyze their performance.
  - The ideal searching algorithm will make the least possible number of comparisons to locate the desired data.
  - Two separate performance analyses are normally done, one for successful search and another for unsuccessful search.

2

Searching: An Example

Unsuccessful Search: search(45) → **NOT_FOUND**

Successful Search: search(12) → **4**



3

Linear Search

- **Linear Search:** A linear search simply examines each item in the search pool, one at a time, until either the target is found or until the pool is exhausted.
- This approach does not assume the items in the search pool are in any particular order
- We just need to be able to examine each element in turn (in a linear fashion)
- It's fairly easy to understand, but not very efficient

4

Linear Search: Performance

- We analyze the successful and unsuccessful searches separately.
- We count how many times the search value is compared against the array elements.
- **Successful Search**
  - **Best Case :** 1 comparison
  - **Worst Case:** N comparisons (N – array size)
- **Unsuccessful Search**
  - **Best Case:**
  - **Worst Case:** N comparisons

5

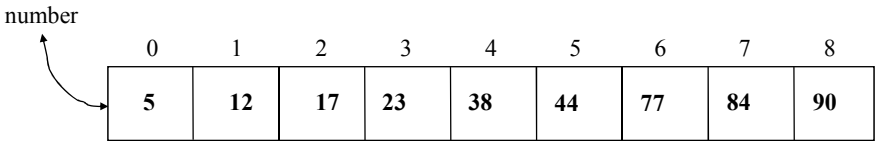
Binary Search

- If the search pool is sorted, then we can be more efficient than a linear search
- A binary search eliminates large parts of the search pool with each comparison
- Instead of starting the search at one end, we begin in the middle
- If the target isn't found, we know that if it is in the pool at all, it is in one half or the other
- We can then jump to the middle of that half, and continue similarly

6

Binary Search: An Example

- If the array is sorted, then we can apply the binary search technique.



- The basic idea is straightforward. First search the value in the middle position. If X is less than this value, then search the middle of the left half next. If X is greater than this value, then search the middle of the right half next. Continue in this manner.

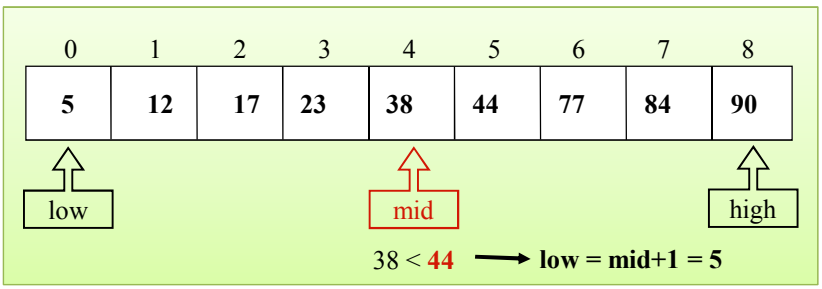
7

Binary Search: An Example

	low	high	mid
#1	0	8	4

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



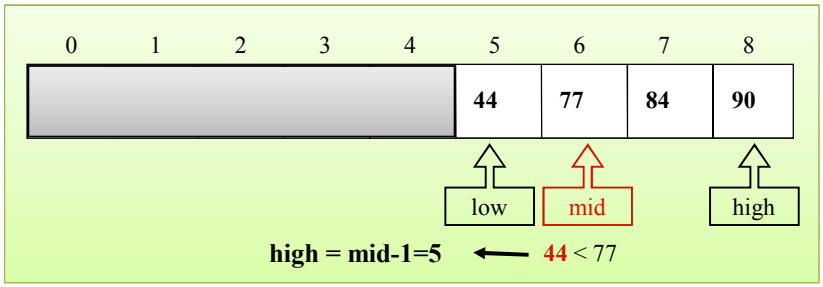
8

Binary Search: An Example

	low	high	mid
#1	0	8	4
#2	5	8	6

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



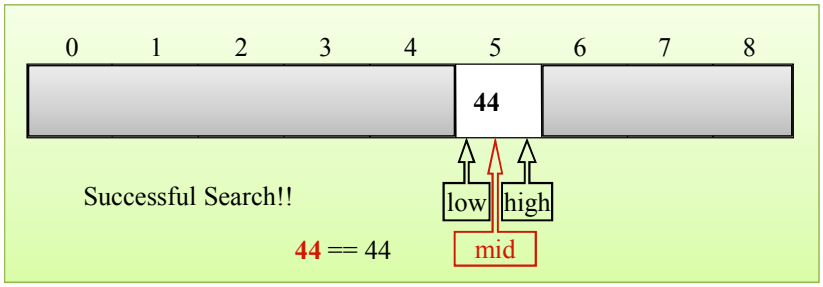
9

Binary Search: An Example

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5

search(44)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



10

Binary Search: Performance

- **Successful Search**
  - **Best Case:** 1 comparison
  - **Worst Case:** $\log_2 N$ comparisons
- **Unsuccessful Search**
  - **Best Case :**
  - **Worst Case :** $\log_2 N$ comparisons
- Since the portion of an array to search is cut into half after every comparison, we compute how many times the array can be divided into halves.

11

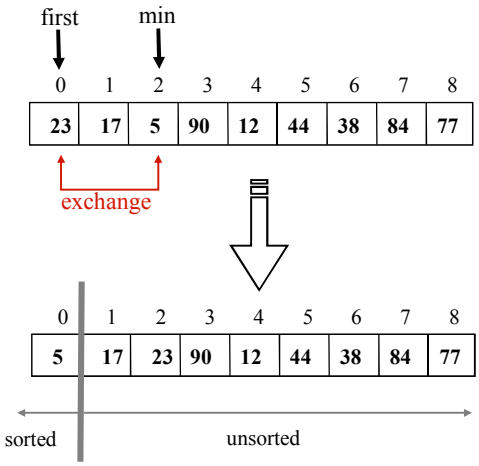
Sorting

- **Sorting** is the process of arranging a group of items into a defined order based on particular criteria, e.g. arranging Person information in ascending order of age.
- Here's the problem statement:
  - **Given an array of N integer values, arrange the values into ascending order.**
- The performance of the sorting algorithm depend on the number of comparisons.
  - **The ideal sorting algorithm will make the least possible number of comparisons to arrange data in a designated order.**
- There are different types of the sorting algorithm such as, **Selection Sort, Bubble Sort, Insertion Sort, Quick Sort and Merge Sort.**

12

Selection Sort

1. Find the smallest element in the list.
2. Exchange the element in the first position and the smallest element. Now the smallest element is in the first position.
3. Repeat Step 1 and 2 with the list having one less element (i.e., the smallest element is discarded from further processing).



This is the result of one pass.

13

Selection Sort

Pass #	0	1	2	3	4	5	6	7	8
1	5	17	23	90	12	44	38	84	77
2	5	12	23	90	17	44	38	84	77
3	5	12	17	90	23	44	38	84	77
●●●									
7	5	12	17	23	38	44	77	84	90
8	5	12	17	23	38	44	77	84	90

sorted

Result AFTER one pass is completed.

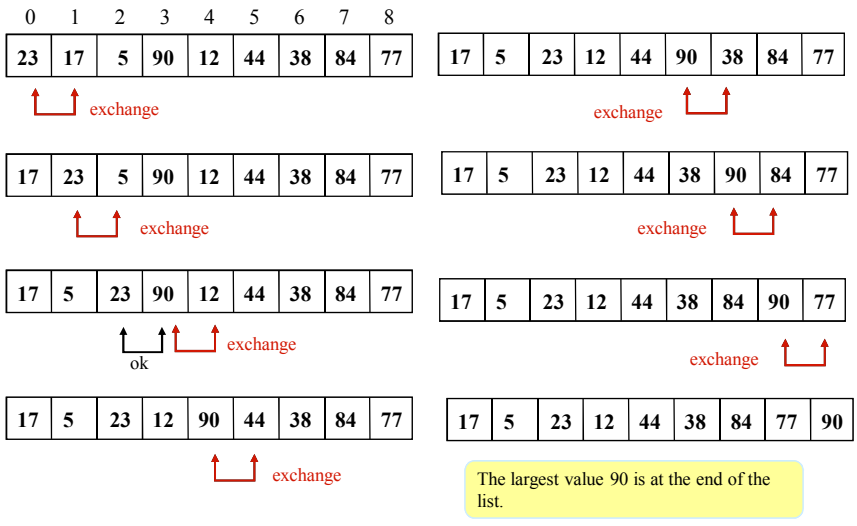
14

Bubble Sort

- With the selection sort, we make one exchange at the end of one pass.
- The bubble sort improves the performance by making more than one exchange during its pass.
- By making multiple exchanges, we will be able to move more elements toward their correct positions using the same number of comparisons as the selection sort makes.
- The key idea of the bubble sort is to make pair wise comparisons and exchange the positions of the pair if they are out of order.

15

Bubble Sort



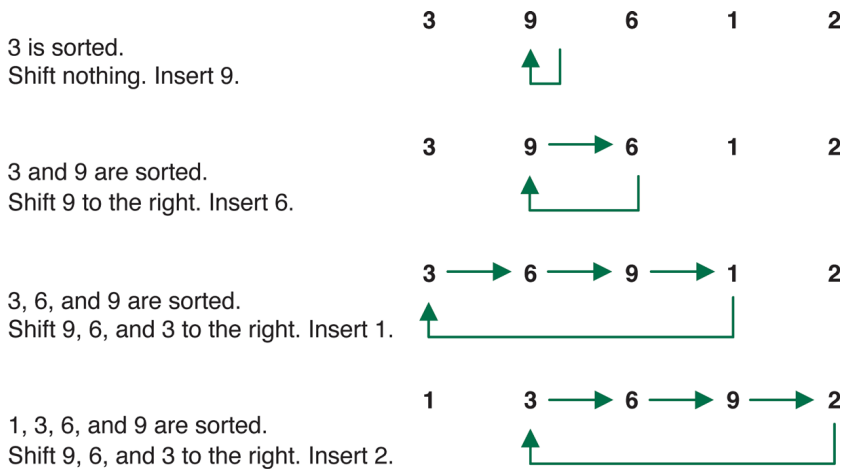
16

Insertion Sort

- Insertion sort orders a list of values by repetitively inserting a particular value into a sorted subset of the list
- More specifically:
  - consider the first item to be a sorted sublist of length **1**
  - insert the second item into the sorted sublist, shifting the first item if needed
  - insert the third item into the sorted sublist, shifting the other items as needed
  - repeat until all values have been inserted into their proper positions

17

Insertion Sort



18

Quick Sort

- **Quick sort** orders a list of values by partitioning the list around one element, then sorting each partition
- More specifically:
  - choose one element in the list to be the partition element
  - organize the elements so that all elements less than the partition element are to the left and all greater are to the right
  - apply the quick sort algorithm (recursively) to both partitions
- The choice of the partition element is arbitrary
- For efficiency, it would be nice if the partition element divided the list roughly in half

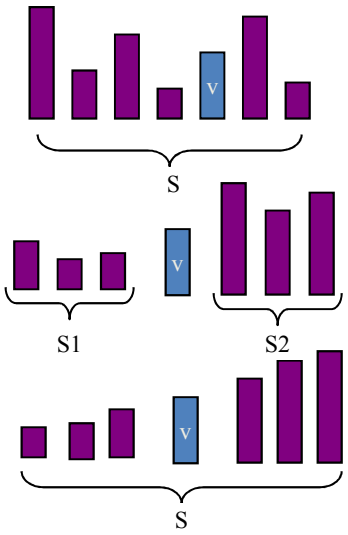
19

Quick sort: An Example

- **Divide step:**
  - Pick any element (**pivot**) v in S
  - Partition S – {v} into two disjoint groups

    S1 = {x ∈ S – {v} | x <= v}

    S2 = {x ∈ S – {v} | x ≥ v}
- **Conquer step**: recursively sort S1 and S2
- **Combine step:** the sorted S1 (by the time returned from recursion), followed by v, followed by the sorted S2 (i.e., nothing extra needs to be done)



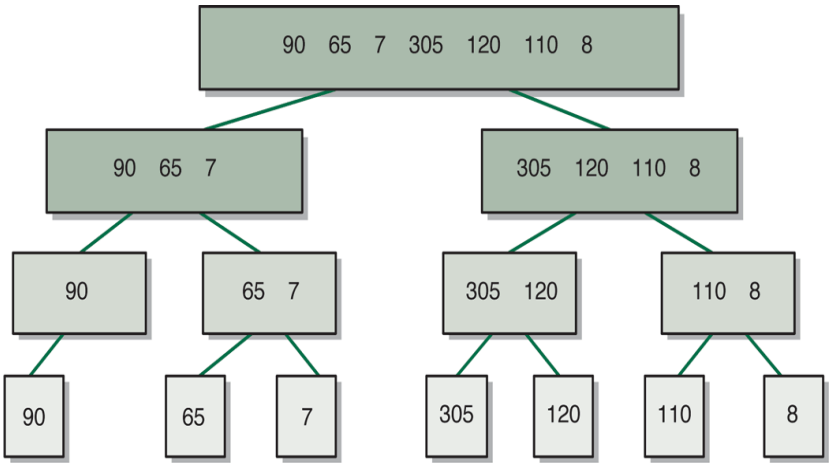
20

Merge Sort

- Merge sort orders a list of values by recursively dividing the list in half until each sub-list has one element, then recombining
- More specifically:
  - divide the list into two roughly equal parts
  - recursively divide each part in half, continuing until a part contains only one element
  - merge the two parts into one sorted list
  - continue to merge parts as the recursion unfolds

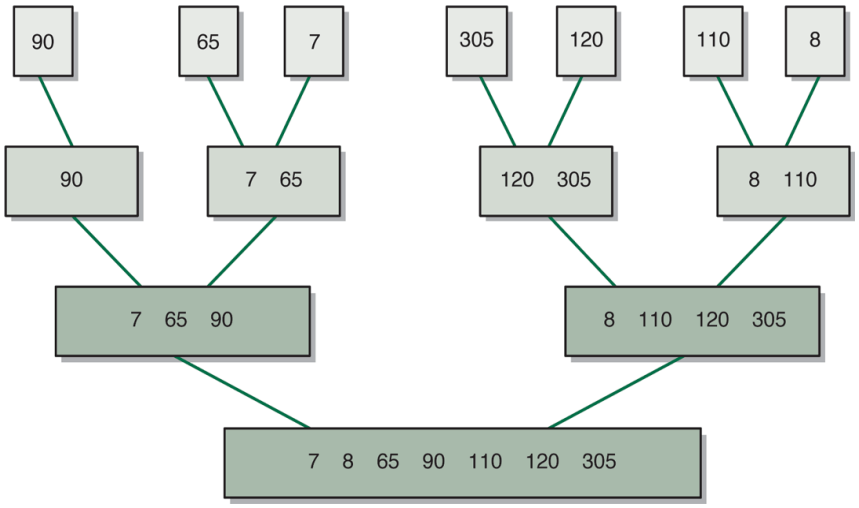
21

Merge Sort: An Example



22

Merge Sort: An Example



23

The End

24